Day Final

Lab 117 · Offloading RoCEv2 ICRC from Tofino Switch to DPU

OpenACC

More Science, Less Programming

Lab 117 @NYCU



- 洪祥銘



- 管培勛



- 黃哲宇



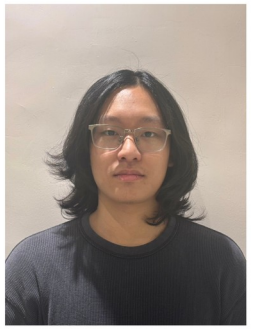
- 江宸安



- 林瀚璿



- 李知恆



- 張慕天



- 陳靖昀

Mentor: Richard @NVIDIA

OpenACC More Science, Less Programming
OPEN HACKATHONS
NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing
NVIDIA

AI training needs one-to-many RDMA

As distributed AI training scales out, the network becomes the bottleneck. After aggregation, the same data must reach many nodes: a natural fit for multicast.

RDMA RC has no multicast

One RC connection per receiver: the sender sends every message 5 times.



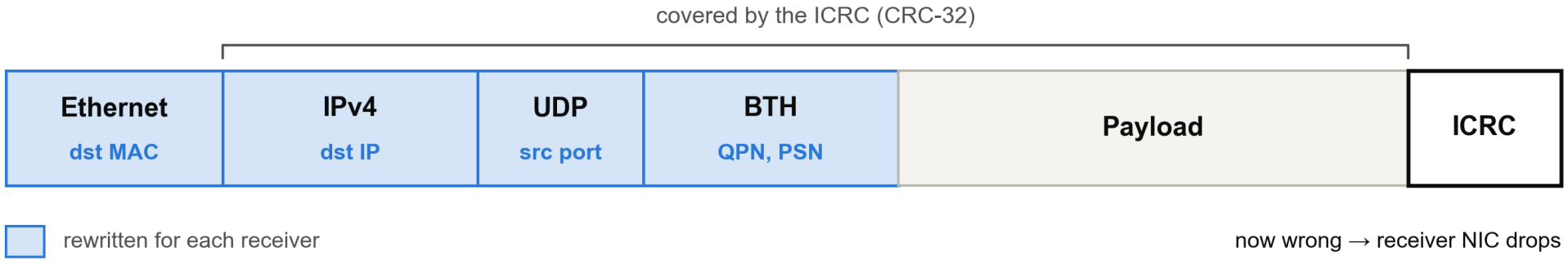
Prior work: in-network replication

P4CE, Cepheus: the sender sends once; the switch copies each packet and rewrites it for its receiver.



But rewriting a packet breaks its ICRC.

Rewriting a packet breaks its ICRC



THE SWITCH

Can’t recompute it

A Tofino pipeline can’t run a CRC over a payload of any length.

OPTION 1

Turn ICRC off

Line rate, but no end-to-end integrity check. Some prior work does this.

OPTION 2

Detour to a CPU

ICRC stays on, but every copy goes out to a CPU and back: packet-in/out.

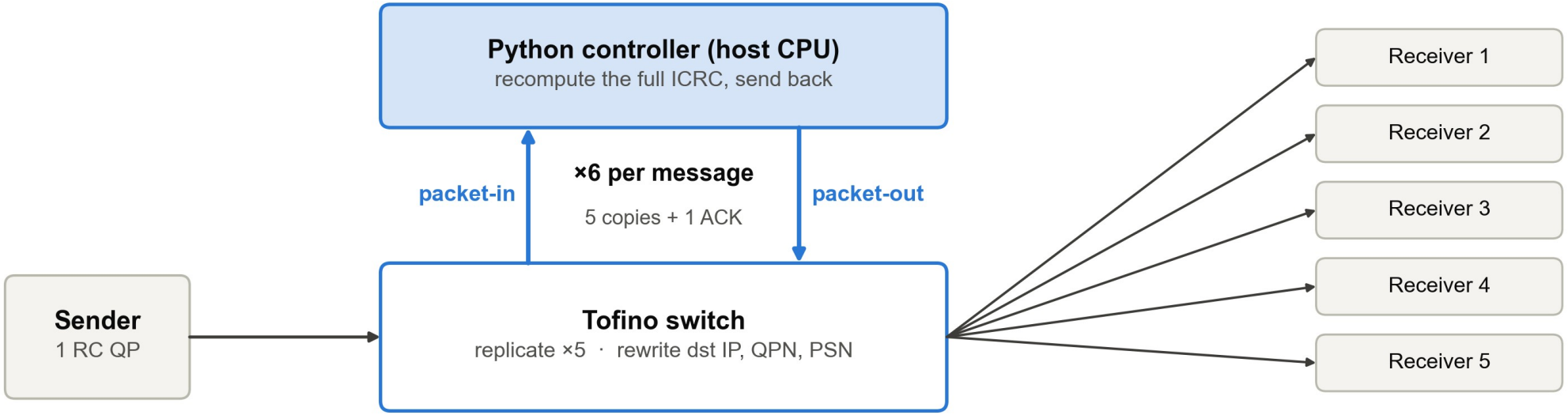
OpenACC More Science, Less Programming
OPEN HACKATHONS
NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing
NVIDIA

Fix the ICRC on the DPU, inline

We move replication and rewriting from the P4 switch to a BlueField-3 DPU. One Arm worker handles each packet in one pass:

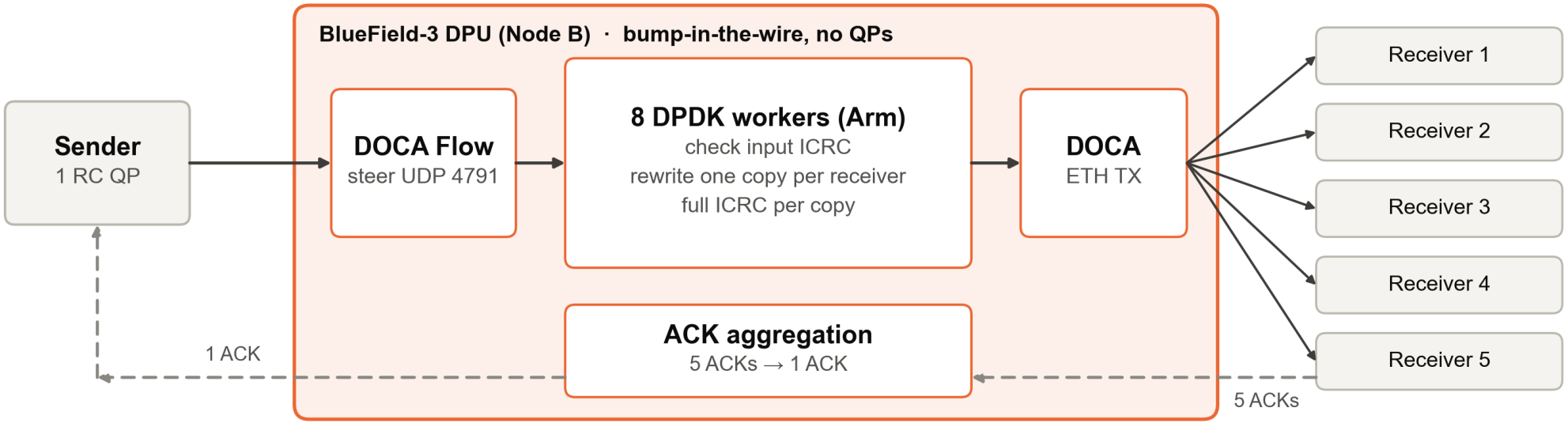


P4 Baseline

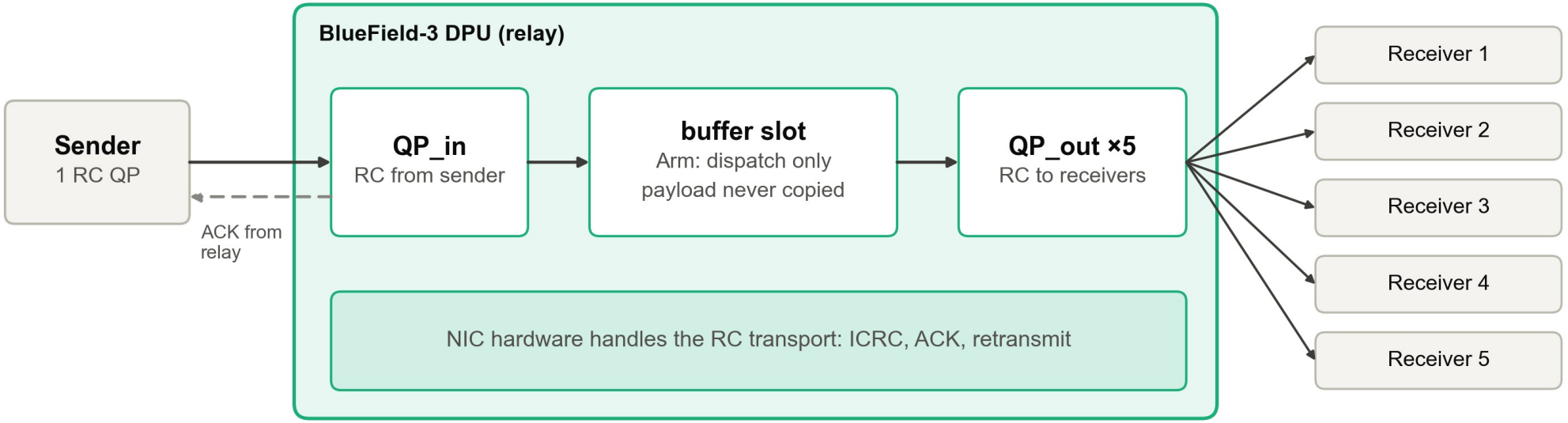


OpenACC More Science, Less Programming
OPEN HACKATHONS
NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing
NVIDIA

Offloading on DPU



Why not using RDMA hardware?



Fast

- The NIC runs RC, ICRC included

But it is just unicast, moved onto the DPU

- To use the RC hardware, the DPU ends the sender's connection and opens one to each receiver
- Not end-to-end: the sender's ACK means the DPU got the data, not the receivers
- State grows: one RC connection per receiver, per group

Result

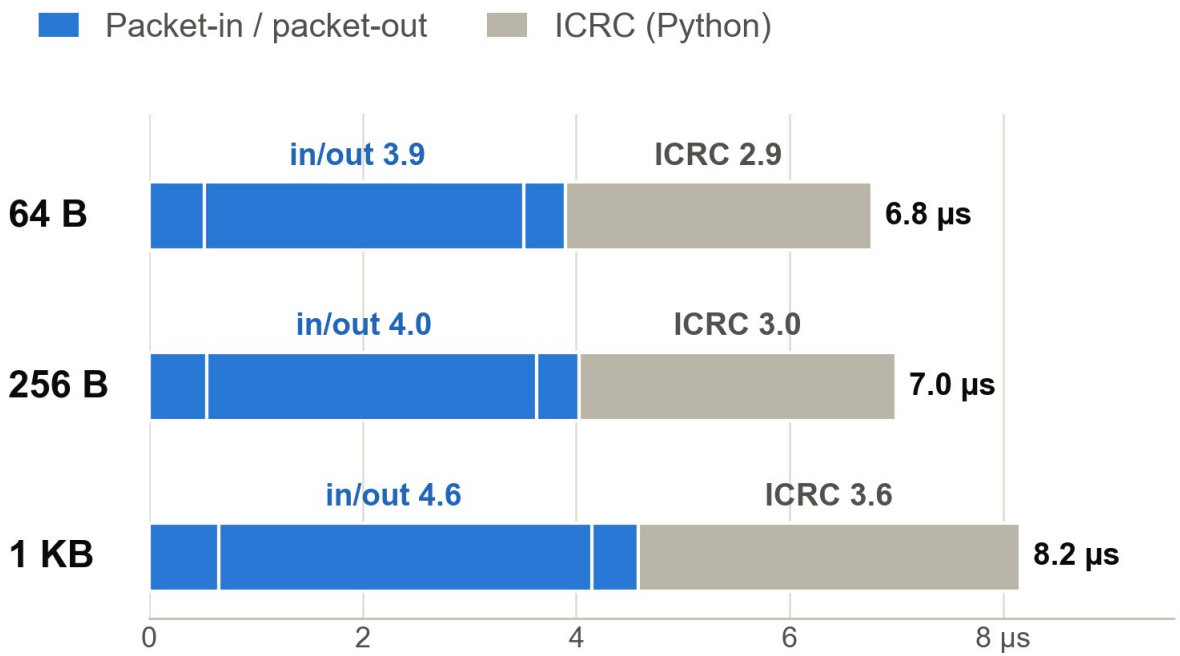
Payload	P4 baseline	DPU	P4 / DPU
64 B	≥ 30.2 µs	≤ 5.2 µs	≥ 5.8×
256 B	≥ 31.2 µs	≤ 10.1 µs	≥ 3.0×
1 KB	≥ 36.5 µs	≤ 27.8 µs	≥ 1.3×
Average	**≥ 32.6 µs**	**≤ 14.4 µs**	≥ 2.3×

2.3×

less time per message on DPU

on average over 64 B–1 KB

Each P4 trip: packet-in/out costs more than the ICRC



Per trip through the controller:

3.9–4.6 µs

packet-in / packet-out

2.9–3.6 µs

ICRC

OpenACC More Science, Less Programming
OPEN HACKATHONS
NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing
NVIDIA

Result summary

2.3×

less time per message on DPU

Average over 64 B–1 KB:
P4 ≥ 32.6 µs, DPU ≤ 14.4 µs

2.0–4.8×

less packet handling, ICRC excluded

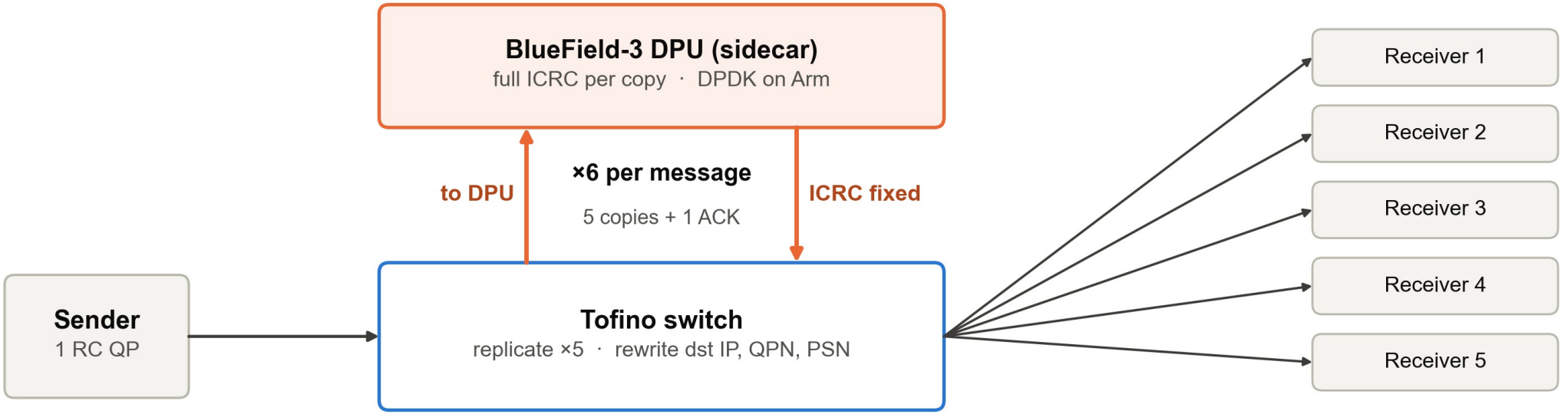
P4 ≥ 15.9–18.6 µs, DPU ≤ 3.3–8.9 µs

97%

of P4's time is the controller detour

6 trips per message; each spends more on packet-in/out than on the ICRC

Estimate: keep the P4 switch, add a DPU sidecar



Sidecar estimate: ≈ 3.4× less time per message

µs per message	64 B	256 B	1 KB	Average
Switch: copy + out (Tofino, measured)	0.91	0.94	1.08	0.98
DPU: ICRC only, 5 copies (DPU, measured)	1.93	5.33	18.90	8.72
Sidecar: switch + DPU (estimate)	**≈ 2.84**	**≈ 6.27**	**≈ 19.98**	**≈ 9.70**
P4 + controller (measured)	≥ 30.21	≥ 31.24	≥ 36.48	≥ 32.64
P4 / sidecar	**≈ 10.6×**	**≈ 5.0×**	**≈ 1.8×**	**≈ 3.4×**

≈ 3.4×

less time per message than P4 + controller

P4 32.64 µs / sidecar 9.70 µs, average of 64 B–1 KB

≈ US$5k

for one BlueField-3 card (retail)

ICRC stays on; the work runs on the DPU's own Arm cores

How AI agents sped up our work

Implementation

Built all three designs from our AGENT.md specification

Testing and debugging

Wrote benchmark scripts, read logs and errors, found bottlenecks

Unattended runs

Ran long tests overnight.

Task	Without AI	With AI	Speed-up
Controller ↔ data plane link	30 min	5 min	6×
Packet-in/out handling	1 hr	10 min	6×
Packet check / classification	10 min	5 min	2×
ICRC calculation	2 hr	20 min	6×
RDMA client/server send/recv	3.2 hr	23 min	8.3×
Total	**6.9 hr**	**63 min**	6.5×

6.5×

faster overall

6.9 hr → 63 min across five development tasks

Conclusion and next steps

Fixing the ICRC inline on the DPU removes the controller detour.

With the same full CRC on both sides, the DPU takes 2.3× less time per message, averaged over 64 B–1 KB: at most 14.4 µs, versus at least 32.6 µs on P4.

Every receiver keeps ICRC checking on, and no message was corrupted or reordered.

NEXT

1. Build the sidecar: the P4 switch replicates and rewrites, a DPU fixes the ICRC

2. Scale out: 8 flows at once on 8 DPU workers (of 16 Arm cores) vs. all 4 cores of the Tofino host CPU, **≈ 4.4× throughput**; measure it with the same window, MTU and hosts


Thank You
OpenACC
More Science, Less Programming